

Aula 2 (Módulo 06) — Arquivos, Regex, JSON e CSV

1. Lendo arquivos

A maior parte da automação de Blue Team começa lendo um arquivo de log. Python abre arquivos com `open()`:

```
with open("auth.log", "r") as arquivo:
    for linha in arquivo:
        print(linha.strip()) # .strip() remove o \n do final da linha
```

O `with` garante que o arquivo seja **fechado automaticamente** ao terminar — mesmo se der erro no meio do processamento. É a forma recomendada e mais segura de abrir arquivos em Python.

Para escrever em um arquivo:

```
with open("saida.txt", "w") as arquivo:
    arquivo.write("IP suspeito: 203.0.113.45\n")
```

(`"r"` = ler, `"w"` = escrever/sobrescrever, `"a"` = adicionar ao final sem apagar o que já existia.)

2. Regex (expressões regulares)

Regex é uma linguagem para **descrever padrões de texto** — muito mais poderosa que `grep` simples (Módulo 01, Aula 7), embora a ideia seja a mesma: "encontre no texto algo que combine com este padrão".

```
import re

texto = "Failed password for root from 203.0.113.45 port 41522 ssh2"

match = re.search(r"\d+\.\d+\.\d+\.\d+", texto)
if match:
    print(match.group()) # 203.0.113.45
```

Símbolos essenciais de regex

Símbolo	Significado
<code>\d</code>	um dígito (0-9)
<code>\d+</code>	um ou mais dígitos seguidos
<code>\w</code>	uma letra, número ou <code>_</code>
<code>.</code>	qualquer caractere
<code>*</code>	zero ou mais repetições do anterior
<code>+</code>	uma ou mais repetições do anterior
<code>[abc]</code>	um caractere entre <code>a</code> , <code>b</code> ou <code>c</code>
<code>()</code>	agrupa parte do padrão (útil para extrair só uma parte)

Extraindo todos os IPs de um texto

```
import re

texto = """
Failed password for root from 203.0.113.45 port 41522 ssh2
Failed password for admin from 198.51.100.10 port 33211 ssh2
Accepted publickey for nicolas from 192.0.2.77 port 52211 ssh2
"""

ips = re.findall(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}", texto)
print(ips)
# ['203.0.113.45', '198.51.100.10', '192.0.2.77']
```

`re.findall()` retorna **todas** as ocorrências do padrão, não só a primeira — perfeito para varrer um log inteiro de uma vez.

Por que importa para segurança: essa é a base de qualquer **IOC extractor** (extrator de indicadores de comprometimento) — pegar um texto bruto (log, e-mail de phishing, relatório de threat intel) e extrair automaticamente IPs, domínios, hashes, e-mails. Vamos construir um de verdade na Aula 4.

3. JSON — o formato de dados mais comum em segurança

JSON (*JavaScript Object Notation*) representa dados estruturados como texto — é o formato que a maioria das APIs (Módulo 03, Aula 7), SIEMs e ferramentas de segurança usa para trocar dados.

```
{
  "ip": "203.0.113.45",
  "usuario": "root",
  "tentativas": 12,
  "bloqueado": true,
  "tags": ["brute-force", "ssh"]
}
```

Repare como isso se parece exatamente com um `dict` do Python (Aula 1) — não é coincidência, a estrutura é quase idêntica.

```
import json

# lendo JSON de um arquivo
with open("evento.json", "r") as arquivo:
    evento = json.load(arquivo)
    print(evento["ip"])

# convertendo um dict Python para JSON (e salvando)
dados = {"ip": "203.0.113.45", "tentativas": 12}
with open("saida.json", "w") as arquivo:
    json.dump(dados, arquivo, indent=2)
```

4. CSV — dados em formato de planilha

CSV (*Comma-Separated Values*) representa dados em linhas e colunas, separadas por vírgula — o formato clássico de exportação de planilhas (Excel) e muitos relatórios de segurança.

```
ip,tentativas,bloqueado
203.0.113.45,12,true
198.51.100.10,3,false
```

```
import csv

with open("eventos.csv", "r") as arquivo:
    leitor = csv.DictReader(arquivo) # já lê cada linha como um dict
    for linha in leitor:
        print(linha["ip"], linha["tentativas"])
```

`csv.DictReader` usa a primeira linha do arquivo (o cabeçalho) para nomear as colunas automaticamente — cada linha seguinte vira um `dict`, igual vimos na Aula 1.

5. Conectando tudo

```
open()      → lê o arquivo de log bruto, linha por linha
re (regex)  → extrai padrões específicos de cada linha (IPs, hashes...)
json/csv    → estruturam a saída, prontos para outra ferramenta consumir
              (ou para você mesmo reabrir depois)
```

Esse é, literalmente, o fluxo de um **parser de log** — que vamos construir do início ao fim na próxima aula.

6. Exercício prático

Crie um arquivo de texto `log_teste.txt` com estas três linhas:

```
Failed password for root from 203.0.113.45 port 41522 ssh2
Failed password for admin from 198.51.100.10 port 33211 ssh2
Accepted publickey for nicolas from 192.0.2.77 port 52211 ssh2
```

Depois, escreva um script Python que: 1. Abra esse arquivo. 2. Use regex para extrair o IP de cada linha. 3. Salve o resultado em um arquivo `ips_extraidos.json`, como uma lista de IPs.

(Dica: combine `open()` + `re.findall()` (Aula 2) + `json.dump()` — tudo que já vimos nesta aula.)

7. Recapitulando

- `open()` com `with` lê/escreve arquivos de forma segura.
- Regex (`re`) descreve padrões de texto — `re.findall()` extrai todas as ocorrências, base de qualquer IOC extractor.
- JSON representa dados estruturados (parecido com `dict`); `json.load` / `json.dump` leem e escrevem.
- CSV representa dados em tabela; `csv.DictReader` já entrega cada linha como `dict`.

Próxima aula

Projeto prático: um parser de log SSH de verdade — do arquivo bruto até um relatório de possíveis tentativas de brute force, código completo e funcional.

Fontes recomendadas

- **Python — documentação oficial do módulo `re`**: referência completa de sintaxe de regex em Python. <https://docs.python.org/3/library/re.html>
- **Python — documentação oficial do módulo `json`**: referência completa. <https://docs.python.org/3/library/json.html>
- **Python — documentação oficial do módulo `csv`**: referência completa. <https://docs.python.org/3/library/csv.html>

